



CPEN 314 – Project Report
Design of a Pipelined Mini-RISC-V Processor

Prepared by:

Rebecca Chalouhi (A2310120)

Nancy Dahdah (A2211690)

Mario Makhlouta (A2220007)

Submitted to:

Dr. Rafic Ayoubi

3/12/2025

Table of Contents

1	Introduction:	3
2	Supported Instruction Set:	3
3	Schematic:	4
4	Pipeline Architecture.....	4
4.1	Fetch (IF):	4
4.2	IF/ID Register	5
4.3	Instruction Decode (ID):.....	5
4.4	ID/EX Register	6
4.5	Execute/Memory (EX/MEM):	7
4.6	EX/MEM/WB Register	9
4.7	Write Back (WB):.....	9
5	Module Descriptions.....	10
5.1	Program Counter (PC).....	10
5.2	Instruction Memory	11
5.3	Register File	12
5.4	Immediate Generator	13
5.5	Control Unit	16
5.6	ALU	18
5.7	ALU Control Unit:	19
5.8	Data Memory	21
5.9	Forwarding Unit.....	25
5.10	Hazard Detection Unit.....	26
6	Simulation Results	27
7	Future work:	30
8	Conclusion.....	31

1 Introduction:

This project presents the design and implementation of a fully pipelined Mini-RISC-V processor supporting a selected subset of the RV32I instruction set architecture. The main objective is to reinforce the concepts learned in class, including instruction execution, datapath organization, pipeline stages, hazards, control signal generation, and throughput improvement.

The processor follows the classical 5-stage RISC pipeline — Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). It is implemented in Quartus using SystemVerilog modules and tested with waveform simulations to verify functional correctness.

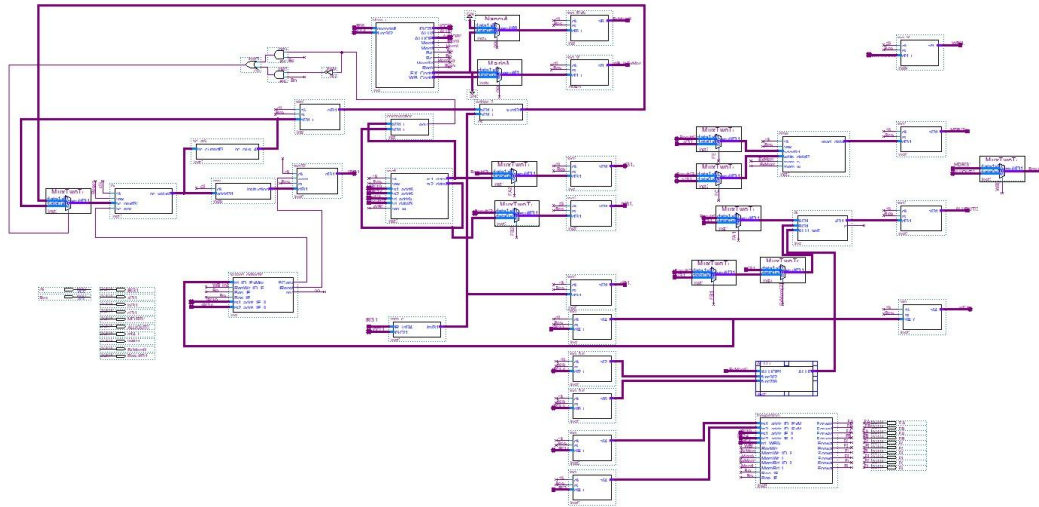
The project confirms key benefits of pipelining, particularly increased instruction throughput and better utilization of hardware resources, while also demonstrating practical handling of hazards and control flow.

2 Supported Instruction Set:

- R-Type: add, sub, and, or, xor, slt, sltu, sll, srl, sra
- I-Type: addi, andi, ori, xori, slti, sltiu, lw, slli, srli, srai
- S-Type: sw
- SB-Type: beq, bne

PS: lw and sw use immediate 0 only.

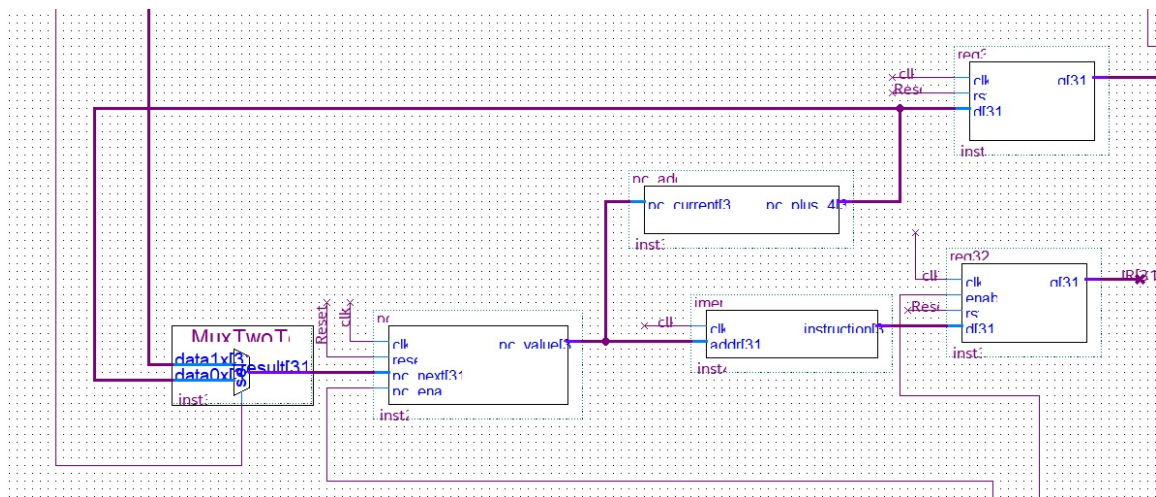
3 Schematic:



4 Pipeline Architecture

4.1 Fetch (IF):

- Fetches the instruction from Instruction memory
- Computes PC + 4



4.2 IF/ID Register

Stores:

- Fetched instruction
- PC

4.3 Instruction Decode (ID):

- Gets the opcode from the IR register and gives it to the main control unit which then outputs the control signals for the following stages
- Inputs rs1 and rs2 from IR and then reads them from the register file
- Inputs IR[31:7] from IR and IGC from the main CU and generates the immediate according to the instruction type.

- #### 4.4 ID/EX Register

- rd, func3 and fun7 from IR
- Gets PC from IF/ID register

4.5 Execute/Memory (EX/MEM):

- ALU performs arithmetic, logical, shift operations
- Calculates branch address
- Evaluates branch conditions using the zero bit (for beq / bne)
- Gets the control signals to the designated spot (IGC, ALUSrc, ALUOP, MemRd, MemWr, Beq, Bne)
- Executes memory instructions:
 - lw → read from data memory
 - sw → write to data memory

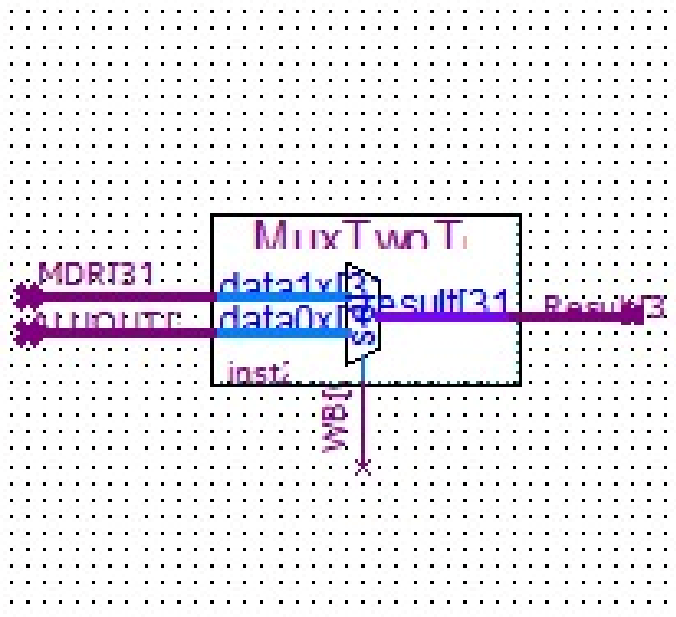
4.6 EX/MEM/WB Register

Stores:

- ALU output (ALUout)
- WB control signals (MemToReg and RegWr)
- Memory output (MDR)
- Gets rd from the ID/EX register

4.7 Write Back (WB):

- Writes the ALU result or memory read data back into rd



5 Module Descriptions

5.1 Program Counter (PC)

```
// Program counter - holds current instruction address
// Can be updated or held during stalls

module pc (
    input logic clk,
    input logic reset,
    input logic [31:0] pc_next,
    input logic pc_enable,
    output logic [31:0] pc_value
);

    always_ff @(posedge clk) begin
        if (reset) begin
            pc_value <= 32'd0;
        end else begin
            if (pc_enable) begin
                pc_value <= pc_next;
            end
        end
    end
endmodule
```

The Program Counter (PC) is a core sequential element responsible for tracking the address of the instruction currently being executed. At every clock cycle, the PC outputs the address used to fetch the next instruction from instruction memory.

In normal operation, the PC increments by 4 to move to the next sequential instruction. However, in cases where a branch instruction is taken, the PC is updated with the computed branch address instead. This behavior allows the processor to support both linear execution flow and conditional branching.

The PC receives its next value through a multiplexer that selects between:

- PC + 4
- Branch address

The PC is updated synchronously at the positive edge of the clock and holds its value during stalls to prevent the pipeline from advancing incorrectly.

Overall, the Program Counter ensures correct control flow in the pipeline by continuously pointing to the address of the next instruction to execute.

5.2 Instruction Memory

```
// instruction memory - stores program instructions
module imem (
    input logic clk,
    input logic [31:0] addr,
    output logic [31:0] instruction
);

    logic [31:0] memory [0:1023];

    initial begin
        // fill with NOPs all IM initially
        for (int i = 0; i < 1024; i++) begin
            memory[i] = 32'h00000013;
        end

        // test program for pipeline
        // reg file starts with x1=1, x2=2, etc
        // data mem starts with mem[0]=1, mem[1]=2, etc

        // basic ADDI tests
        memory[0] = 32'h00100093; // ADDI x1, x0, 1      : x1 = 0 + 1 = 1 (overwrite initial x1=1)
        memory[1] = 32'h00200113; // ADDI x2, x0, 2      : x2 = 0 + 2 = 2 (overwrite initial x2=2)
        memory[2] = 32'h00300193; // ADDI x3, x0, 3      : x3 = 0 + 3 = 3 (overwrite initial x3=3)
        memory[3] = 32'h00400213; // ADDI x4, x0, 4      : x4 = 0 + 4 = 4 (overwrite initial x4=4)

        // forwarding tests
        // case 1: back-to-back instructions using same register
        memory[4] = 32'h002082b3; // ADD x5, x1, x2      : x5 = x1 + x2 = 1 + 2 = 3
        memory[5] = 32'h40528333; // SUB x6, x5, x5      : x6 = x5 - x5 = 0 (FORWARDING: x5 from MEM)

        // case 2: one instruction gap
        memory[6] = 32'h003083b3; // ADD x7, x1, x3      : x7 = x1 + x3 = 1 + 3 = 4
        memory[7] = 32'h00400413; // ADDI x8, x0, 4      : x8 = 0 + 4 = 4 (delay instruction)
        memory[8] = 32'h007484b3; // ADD x9, x9, x7      : x9 = x9 + x7 (FORWARDING: x7 from WB)

        // case 3: both inputs need forwarding
        memory[9] = 32'h00408533; // ADD x10, x1, x4     : x10 = x1 + x4 = 1 + 4 = 5
        memory[10] = 32'h005085b3; // ADD x11, x1, x5     : x11 = x1 + x5 = 1 + 3 = 4
        memory[11] = 32'h00A585b3; // ADD x11, x11, x10   : x11 = x11 + x10 (FORWARDING: both from WB)

        // R-type instruction tests
        memory[12] = 32'h00310633; // ADD x12, x2, x3     : x12 = x2 + x3 = 2 + 3 = 5
        memory[13] = 32'h40c106b3; // SUB x13, x2, x12    : x13 = x2 - x12 = 2 - 5 = -3
        memory[14] = 32'h00317733; // AND x14, x2, x3     : x14 = x2 & x3 = 2 & 3 = 2
        memory[15] = 32'h0031e733; // OR x14, x3, x3      : x14 = x3 | x3 = 3 | 3 = 3
        memory[16] = 32'h00314733; // XOR x14, x2, x3     : x14 = x2 ^ x3 = 2 ^ 3 = 1

        // I-type logic instructions
        memory[17] = 32'h00f0f013; // ANDI x0, x1, 15     : x0 = x1 & 15 = 1 & 15 = 1 (x0 stays 0!)
        memory[18] = 32'h00f0e113; // ORI x2, x1, 15      : x2 = x1 | 15 = 1 | 15 = 15
        memory[19] = 32'h00f14113; // XORI x2, x2, 15     : x2 = x2 ^ 15 = 15 ^ 15 = 0

        // I-type shift instructions
        memory[20] = 32'h00109093; // SLLI x1, x1, 1      : x1 = x1 << 1 = 1 << 1 = 2
        memory[21] = 32'h4010p093; // SRLI x1, x1, 1      : x1 = x1 >> 1 = 2 >> 1 = 1 (logical)
        memory[22] = 32'h40115093; // SRAI x1, x1, 1      : x1 = x1 >>> 1 = 1 >>> 1 = 0 (arithmetic, x1 is positive)

        // R-type shift instructions
        memory[23] = 32'h001191b3; // SLL x3, x3, x1      : x3 = x3 << x1[4:0] = 3 << 1 = 6
        memory[24] = 32'h0011d1b3; // SRL x3, x3, x1      : x3 = x3 >> x1[4:0] = 6 >> 1 = 3 (logical)
        memory[25] = 32'h4011d1b3; // SRA x3, x3, x1      : x3 = x3 >>> x1[4:0] = 3 >>> 1 = 1 (arithmetic)

        // I-type comparison instructions
        memory[26] = 32'h0030a013; // SLTI x0, x1, 3      : x0 = (signed x1 < 3) ? 1 : 0 (x0 stays 0!)
        memory[27] = 32'h0030a113; // SLTI x2, x1, 3      : x2 = (signed x1 < 3) ? 1 : 0 = 1 (if x1 < 3)
        memory[28] = 32'h0030b113; // SLTIU x2, x1, 3     : x2 = (unsigned x1 < 3) ? 1 : 0 = 1 (if x1 < 3)
        memory[29] = 32'h00a0a113; // SLTI x2, x1, 10     : x2 = (signed x1 < 10) ? 1 : 0 = 1 (if x1 < 10)

        // R-type comparison instructions
        memory[30] = 32'h0011a133; // SLT x2, x3, x1      : x2 = (signed x3 < x1) ? 1 : 0 = (3 < 1) ? 1 : 0 = 0
        memory[31] = 32'h0011b133; // SLTU x2, x3, x1     : x2 = (unsigned x3 < x1) ? 1 : 0 = (3 < 1) ? 1 : 0 = 0
        memory[32] = 32'h0031a1b3; // SLT x3, x3, x3      : x3 = (signed x3 < x3) ? 1 : 0 = 0 (always false)
        memory[33] = 32'h0021a2b3; // SLT x5, x3, x2      : x5 = (signed x3 < x2) ? 1 : 0 = (3 < 0) ? 1 : 0 = 0

        // setup for memory tests
        memory[34] = 32'h00000113; // ADDI x2, x0, 0      : x2 = 0 (address for memory access, overwrites x2)

        // load-use hazard tests (HDL should stall these)
        // case 1: LW then use result immediately
        memory[35] = 32'h0000a0b3; // LW x1, 0(x2)        : x1 = mem[x2+0] = mem[0] = 1 (load from memory)
        memory[36] = 32'h001081b3; // ADD x3, x1, x1      : x3 = x1 + x1 (HAZARD: needs x1, HDU stall 1 cycle)
        // Expected: After stall, x3 = 1 + 1 = 2 (forwarding from WB)

        // case 2: LW then use in different register
        memory[37] = 32'h0000a103; // LW x2, 0(x2)        : x2 = mem[x2+0] = mem[0] = 1 (overwrites x2=0)
        memory[38] = 32'h402106b3; // SUB x13, x2, x2     : x13 = x2 - x2 (HAZARD: needs x2, HDU stall 1 cycle)
        // Expected: After stall, x13 = 1 - 1 = 0 (forwarding from WB)

        // case 3: LW result used as rs2
        memory[39] = 32'h00100113; // ADDI x2, x0, 1      : x2 = 1 (setup address)
        memory[40] = 32'h0000a203; // LW x4, 0(x2)        : x4 = mem[x2+0] = mem[1] = 2
        memory[41] = 32'h00418333; // ADD x6, x3, x4      : x6 = x3 + x4 (HAZARD: needs x4, HDU stall 1 cycle)
        // Expected: After stall, x6 = 2 + 2 = 4 (forwarding from WB, x3=2 from previous)
    end
endmodule
```

The Instruction Memory stores the machine-code instructions that the processor executes. It is a read-only memory (ROM-like structure) that outputs the instruction located at the address provided by the Program Counter (PC). During each clock cycle, the Instruction Fetch (IF) stage uses the PC value to read the corresponding 32-bit instruction from this memory.

The memory interface is simple:

- Input: PC (instruction address)
- Output: 32-bit instruction word

The fetched instruction is forwarded to the IF/ID pipeline register IR along with PC + 4, allowing subsequent stages to decode and execute it. The Instruction Memory therefore plays a crucial role in providing the pipeline with a continuous stream of instructions and maintaining the correct flow of program execution.

5.3 Register File

```
// Register file - 32 registers, x0 hardwired to zero
// Can read from two registers and write to one per cycle

module regfile (
    input logic clk,
    input logic reset,

    // read addresses and data
    input logic [4:0] rs1_addr,
    output logic [31:0] rs1_data,
    input logic [4:0] rs2_addr,
    output logic [31:0] rs2_data,

    // write port
    input logic [4:0] rd_addr,
    input logic [31:0] rd_data,
    input logic reg_write
);

    // only store x1-x31, x0 is always zero
    logic [31:0] registers [1:31];

    // init registers for testing
    initial begin
        for (int i = 1; i < 32; i++) begin
            registers[i] = 32'(i);
        end
    end

    // reads are combinational
    // x0 always returns zero
    assign rs1_data = (rs1_addr == 5'b00000) ? 32'd0 : registers[rs1_addr];
    assign rs2_data = (rs2_addr == 5'b00000) ? 32'd0 : registers[rs2_addr];
```

```

// writes happen on clock edge
always_ff @(posedge clk) begin
    if (reset) begin
        for (int i = 1; i < 32; i++) begin
            registers[i] <= 32'd0;
        end
    end else begin
        // can't write to x0
        if (reg_write && (rd_addr != 5'b00000)) begin
            registers[rd_addr] <= rd_data;
        end
    end
end
endmodule

```

The Register File stores the 32 general-purpose registers defined by the RISC-V architecture. It supports two simultaneous read operations, and one write operation per clock cycle, enabling efficient operand access for the pipeline.

The Register File receives the register indices `rs1`, `rs2`, and `rd` from the instruction during the ID stage. Based on these indices:

- It outputs the values of `rs1` and `rs2` to be used by the ALU or forwarded to later stages.
- It writes data into `rd` during the WB stage when the `RegWrite` control signal is enabled.

A key architectural rule enforced by the Register File is that register `x0` is hardwired to zero. Any read from `x0` always returns 0, and any write attempt to `x0` is ignored. This behavior ensures compliance with the RISC-V standard and simplifies many datapath operations.

The Register File plays a critical role in the pipeline by providing operands for arithmetic, logical, memory, and branch instructions, while synchronizing write-back results from the MEM/WB pipeline stage.

5.4 Immediate Generator

Generates immediate values for:

The Immediate Generator is responsible for extracting and constructing the immediate values used by various RISC-V instruction formats. It takes the `IR[31:7]` instruction word as input and produces a correctly sign-extended immediate based on the instruction type (I-type, S-type, or B-type).

Different instruction formats store immediate bits in different positions. The Immediate Generator reconstructs these fields by selecting the necessary bits, reordering them when needed, and applying sign extension so the immediate can be used correctly by the ALU.

The generated immediate is used for operations such as:

- ALU immediate instructions (addi, andi, ori, xori, slti, etc.)
- Branch target offset computation (beq, bne)

Because immediate values are needed during the Decode (ID) stage, the Immediate Generator ensures that each instruction produces the correct operand width and format before being passed to the ID/EX/ pipeline register C. This makes it a crucial component for enabling arithmetic-on-immediates, memory addressing, and branch control flow.

```
// immediate generator - extracts and sign-extends immediate from instruction
module imm_gen (
    input logic [24:0] IR_in,    // IR[31:7] compressed
    input logic [1:0] IGC,      // 00=I, 01=S, 10=B
    output logic [31:0] Im
);

    logic IR31;
    logic [5:0] IR30_25;
    logic [3:0] IR11_8;
    logic IR7;
    logic [11:0] IR31_20;
    logic [6:0] IR31_25;
    logic [4:0] IR11_7;

    assign IR31 = IR_in[24];
    assign IR30_25 = IR_in[23:18];
    assign IR11_8 = IR_in[4:1];
    assign IR7 = IR_in[0];
    assign IR31_20 = {IR_in[24:13]};
    assign IR31_25 = {IR_in[24:18]};
    assign IR11_7 = {IR_in[4:0]};

    logic [31:0] imm_i;
    logic [31:0] imm_s;
    logic [31:0] imm_b;

    // I-type immediate
    always_comb begin
        imm_i[11:0] = IR31_20;
        imm_i[31:12] = {20{IR31}};
    end
end
```

```

// S-type immediate
always_comb begin
    imm_s[4:0] = IR11_7;
    imm_s[11:5] = IR31_25;
    imm_s[31:12] = {20{IR31}};
end

// B-type immediate
always_comb begin
    imm_b[0] = 1'b0;
    imm_b[4:1] = IR11_8;
    imm_b[10:5] = IR30_25;
    imm_b[11] = IR7;
    imm_b[12] = IR31;
    imm_b[31:13] = {19{IR31}};
end

always_comb begin
    unique case (IGC)
        2'b00: Im = imm_i;
        2'b01: Im = imm_s;
        2'b10: Im = imm_b;
        default: Im = imm_i;
    endcase
end

endmodule

```

The immediate generator extracts, formats, and sign-extends the required immediate value from the instruction for I-type, S-type, and B-type RISC-V formats. Since the datapath only provides bits IR[31:7] into this unit, the module reconstructs the appropriate fields by slicing and rearranging the compressed instruction input (IR_in). For each immediate type, the unit assembles the immediate according to the RISC-V encoding rules. The I-type immediate (imm_i) is formed from bits [31:20], then sign-extended from bit 31. The S-type immediate (imm_s) combines bits [31:25] and [11:7], again with proper sign extension. The B-type immediate (imm_b) reconstructs the branch offset by placing bits [11], [4:1], [10:5], and the sign bit in the correct positions, sets bit 0 to zero (because branch offsets are word-aligned), and sign-extends from bit 12. The control signal IGC selects which immediate to output, allowing the module to generate the proper value for arithmetic immediates (I-type), store offsets (S-type), and branch displacements (B-type). This module centralizes immediate extraction and ensures that all immediates are properly sign-extended to 32 bits before entering the execution stage.

5.5 Control Unit

// main control unit - decodes instruction opcode and generates control signals

```
module Main_CU (
    input logic [6:0] opcode,
    input logic [2:0] func3,

    // individual control signals
    output logic [1:0] IGC,
    output logic ALUSrc,
    output logic [1:0] ALUOP,
    output logic MemRd,
    output logic MemWr,
    output logic Beq,
    output logic Bne,
    output logic MemToReg,
    output logic RegWr,

    // bundled for pipeline
    output logic [6:0] EX_Control,
    output logic [1:0] WB_Control
);
```

```
    logic [1:0] IGC_int;
    logic ALUSrc_int;
    logic [1:0] ALUOP_int;
    logic MemRd_int;
    logic MemWr_int;
    logic Beq_int;
    logic Bne_int;
    logic MemToReg_int;
    logic RegWr_int;
```

```
always_comb begin
    case (opcode)
        // R-type
        7'b0110011: begin
            IGC_int    = 2'b00;
            ALUSrc_int = 1'b0;
            ALUOP_int  = 2'b00;
            MemRd_int  = 1'b0;
            MemWr_int  = 1'b0;
            Beq_int    = 1'b0;
            Bne_int    = 1'b0;
            MemToReg_int = 1'b0;
            RegWr_int  = 1'b1;
        end

        // I-type (not LW)
        7'b0010011: begin
            IGC_int    = 2'b00;
            ALUSrc_int = 1'b1;
            ALUOP_int  = 2'b10;
            MemRd_int  = 1'b0;
            MemWr_int  = 1'b0;
            Beq_int    = 1'b0;
            Bne_int    = 1'b0;
            MemToReg_int = 1'b0;
            RegWr_int  = 1'b1;
        end

        // LW
        7'b0000011: begin
            IGC_int    = 2'b00;
            ALUSrc_int = 1'b1;
            ALUOP_int  = 2'b00;
            MemRd_int  = 1'b1;
            MemWr_int  = 1'b0;
            Beq_int    = 1'b0;
            Bne_int    = 1'b0;
            MemToReg_int = 1'b1;
            RegWr_int  = 1'b1;
        end
    end
```

```

// SW
7'b0100011: begin
    IGC_int    = 2'b01;
    ALUSrc_int = 1'b1;
    ALUOP_int  = 2'b00;
    MemRd_int  = 1'b0;
    MemWr_int  = 1'b1;
    Beq_int    = 1'b0;
    Bne_int    = 1'b0;
    MemToReg_int = 1'b0;
    RegWr_int  = 1'b0;
end

// branch
7'b1100011: begin
    IGC_int    = 2'b10;
    ALUSrc_int = 1'b0;
    ALUOP_int  = 2'b01;
    MemRd_int  = 1'b0;
    MemWr_int  = 1'b0;
    Beq_int    = (func3 == 3'b000) ? 1'b1 : 1'b0;
    Bne_int    = (func3 == 3'b001) ? 1'b1 : 1'b0;
    MemToReg_int = 1'b0;
    RegWr_int  = 1'b0;
end

default: begin
    IGC_int    = 2'b00;
    ALUSrc_int = 1'b0;
    ALUOP_int  = 2'b00;
    MemRd_int  = 1'b0;
    MemWr_int  = 1'b0;
    Beq_int    = 1'b0;
    Bne_int    = 1'b0;
    MemToReg_int = 1'b0;
    RegWr_int  = 1'b0;
end
endcase
end

assign IGC = IGC_int;
assign ALUSrc = ALUSrc_int;
assign ALUOP = ALUOP_int;
assign MemRd = MemRd_int;
assign MemWr = MemWr_int;
assign Beq = Beq_int;
assign Bne = Bne_int;
assign MemToReg = MemToReg_int;
assign RegWr = RegWr_int;

assign EX_Control = {Bne_int, Beq_int, MemWr_int, MemRd_int, ALUSrc_int, ALUOP_int};
assign WB_Control = {RegWr_int, MemToReg_int};
endmodule

```

The Control Unit is responsible for decoding the instruction and generating all necessary control signals that guide the datapath components in each pipeline stage. Based on the opcode (and, when necessary, funct3 and funct7), the Control Unit determines the instruction type (R-format, I-format, S-format, or B-format) and sets the appropriate control outputs for the ALU, register file, memory, and branch logic.

These control signals dictate how the processor behaves for each instruction by enabling or disabling specific operations such as register writing, memory access, or ALU input selection. For example, ALUSrc selects between a register operand or an immediate value, MemToReg chooses whether the value written back to the register file comes from the memory or from the ALU, and RegWr enables register writes during the WB stage. Branch signals (Beq and Bne) activate the branch comparison logic in the EX stage.

The table below summarizes the generated control signals for each supported instruction category. These values ensure correct datapath operation for arithmetic, logic, load/store, and branch instructions.

Main C.U.	IGC	ALUSrc	ALUOP	MemRd	MemWr	Beq	Bne	MemToReg	RegWr
R-format (all)	xx	0	00	0	0	0	0	0	1
I-format (not lw)	00	1	10	0	0	0	0	0	1
I-format (lw)	00	x	xx	1	0	0	0	1	1
S-format (sw)	01	x	xx	0	1	0	0	x	0
Beq	10	0	01	0	0	1	0	x	0
Bne	10	0	01	0	0	0	1	x	0

NB: in system Verilog we cannot use x (don't care), we assigned them to be zeros.

5.6 ALU

```
// ALU module - handles all arithmetic and logic operations
// Inputs: two 32-bit operands and 4-bit operation select
// Outputs: 32-bit result and zero flag

module alu (
    input logic [31:0] A,      // first operand
    input logic [31:0] B,      // second operand
    input logic [3:0] ALU_sel, // operation selector
    output logic [31:0] y,     // result output
    output logic z             // zero flag
);

    logic [31:0] result;

    // main ALU operations
    always_comb begin
        case (ALU_sel)
            4'b0000: result = A + B;           // add
            4'b0001: result = A - B;           // subtract
            4'b0010: result = A << B[4:0];     // shift left
            4'b0011: result = A ^ B;           // xor
            4'b0100: result = A >> B[4:0];     // shift right logical
            4'b0101: result = $signed(A) >>> B[4:0]; // shift right arithmetic
            4'b0110: result = A | B;           // or
            4'b0111: result = A & B;           // and
            4'b1000: result = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0; // set less than (signed)
            4'b1001: result = (A < B) ? 32'd1 : 32'd0; // set less than (unsigned)
            default: result = 32'd0;
        endcase
    end

    assign y = result;
    assign z = (result == 32'd0) ? 1'b1 : 1'b0;
endmodule
```

The Arithmetic Logic Unit (ALU) is responsible for executing all arithmetic, logical, comparison, and shift operations required by the processor. It receives two input operands (either from the register file or the immediate generator, depending on ALUSrc) and performs an operation based on the ALU control signal (ALUOp) generated by the control unit.

The ALU supports all operations required by the implemented RISC-V subset, including:

- Arithmetic: add, sub

- Logical: and, or, xor
- Shift operations: sll, srl, sra
- Comparison: slt (signed), sltu (unsigned)

In addition to producing the operation result, the ALU also generates status signals such as the **Zero** flag, which is used by branch instructions (beq, bne) to determine whether a branch should be taken.

5.7 ALU Control Unit:

```
// ALU control unit - generates ALU operation code from ALUOP, func3, func7
module ALU_CU (
    input logic [1:0] ALUOP,
    input logic [2:0] func3,
    input logic [6:0] func7,
    output logic [3:0] ALU
);

    always_comb begin
        case (ALUOP)
            // R-type
            2'b00: begin
                if (func3 == 3'b010) begin
                    if (func7 == 7'b0000000) begin
                        ALU = 4'b1000; // SLT
                    end else begin
                        ALU = 4'b0000; // LW/SW, don't care
                    end
                end else begin
                    case ({func7[5], func3})
                        {1'b0, 3'b000}: ALU = 4'b0000; // ADD
                        {1'b1, 3'b000}: ALU = 4'b0001; // SUB
                        {1'b0, 3'b001}: ALU = 4'b0010; // SLL
                        {1'b0, 3'b011}: ALU = 4'b1001; // SLTU
                        {1'b0, 3'b100}: ALU = 4'b0011; // XOR
                        {1'b0, 3'b101}: ALU = 4'b0100; // SRL
                        {1'b1, 3'b101}: ALU = 4'b0101; // SRA
                        {1'b0, 3'b110}: ALU = 4'b0110; // OR
                        {1'b0, 3'b111}: ALU = 4'b0111; // AND
                    default: ALU = 4'b0000;
                endcase
            end
        endcase
    end

    // I-type
    2'b10: begin
        case (func3)
            3'b000: ALU = 4'b0000; // ADDI
            3'b001: begin
                if (func7[5] == 1'b0) begin
                    ALU = 4'b0010; // SLLI
                end else begin
                    ALU = 4'b0000;
                end
            end
        endcase
    end
end
```

```

        3'b010: ALU = 4'b1000; // SLTI
        3'b011: ALU = 4'b1001; // SLTIU
        3'b100: ALU = 4'b0011; // XORI
        3'b101: begin
            if (func7[5] == 1'b0) begin
                ALU = 4'b0100; // SRLI
            end else begin
                ALU = 4'b0101; // SRAI
            end
        end
        3'b110: ALU = 4'b0110; // ORI
        3'b111: ALU = 4'b0111; // ANDI
        default: ALU = 4'b0000;
    endcase
end

// branch
2'b01: begin
    ALU = 4'b0001; // SUB for comparison
end

default: begin
    ALU = 4'b0000;
end
endcase
end
endmodule

```

The ALU Control Unit translates the high-level ALUOp signals from the Main Control Unit and the instruction's funct3/funct7 fields into the specific ALU control code required by the ALU to perform the correct operation.

For R-type instructions, the ALUOp is 00. The ALU Control Unit uses both funct3 and funct7 to uniquely identify arithmetic, logical, and shift operations such as add, sub, xor, sll, slt, and sltu.

For I-type ALU instructions, ALUOp is "10". In this case, the unit relies primarily on funct3 and funct7 only for distinguishing between srli and srai) to produce the corresponding ALU control code.

For branch instructions (beq, bne), ALUOp is "01". The ALU performs a subtraction to compare operands, and the ALU Control Unit generates the appropriate control code used by the branch comparator.

The ALU control signal generated is a compact 4-bit code that directly selects which operation the ALU executes. This ensures correct behavior for all supported arithmetic, logical, shift, load/store, and branch instructions.

By combining opcode-level control (ALUOp) with instruction-level fields (funct3 and funct7), the ALU Control Unit precisely determines the appropriate ALU function required in each cycle of the pipeline.

ALU C.U.	ALUOP	func3	func7	ALU
add	00	000	0000000	0000
sub	00	000	0100000	0001
sll	00	001	0000000	0010
xor	00	100	0000000	0011
srl	00	101	0000000	0100
sra	00	101	0000000	0101
or	00	110	0000000	0110
and	00	111	0000000	0111
slt	00	010	0000000	1000
sltu	00	011	0000000	1001
lw	xx	xxx	xxxxxxx	xxxx
addi	10	000	xxxxxxx	0000
slli	10	001	xxxxxxx	0010
xori	10	100	xxxxxxx	0011
srli	10	101	0000000	0100
srai	10	101	0100000	0101
ori	10	110	xxxxxxx	0110
andi	10	111	xxxxxxx	0111
slti	10	010	xxxxxxx	1000
sltiu	10	011	xxxxxxx	1001
sw	xx	xxx	xxxxxxx	xxxx
beq	01	000	xxxxxxx	0001
bne	01	001	xxxxxxx	0001

5.8 Data Memory

The Data Memory module provides storage for load and store instructions during the MEM stage of the pipeline. It supports read and write operations depending on the control signals generated by the Control Unit.

For load instructions (lw), the Data Memory receives the address from register A in ID/EX/MEM (since EX and MEM are in the same stage) pipeline register and outputs the 32-bit data stored at that location in RF. This read operation is enabled when the MemRd signal is active.

For store instructions (sw), the Data Memory writes the value from the register file passed through the pipeline register B into the specified memory address when the MemWr signal is asserted.

In this project, Data is used to verify correct functioning of lw and sw instructions during simulation. Only the load and store instructions interact with this module, while all other instructions bypass it.

```

// data memory - handles load/store operations
module dmem (
    input logic clk,
    input logic reset,
    input logic [31:0] addr,
    input logic [31:0] write_data,
    input logic mem_read,
    input logic mem_write,
    output logic [31:0] read_data
);

    logic [31:0] memory [0:1023];

    // init memory for testing
    initial begin
        for (int i = 0; i < 1024; i++) begin
            memory[i] = 32'(i + 1);
        end
    end

    always_ff @(posedge clk) begin
        if (reset) begin
            for (int i = 0; i < 1024; i++) begin
                memory[i] <= 32'd0;
            end
        end else begin
            if (mem_write) begin
                memory[addr[11:2]] <= write_data;
            end
        end
    end

    always_ff @(posedge clk) begin
        if (mem_read) begin
            read_data <= memory[addr[11:2]];
        end else begin
            read_data <= 32'd0;
        end
    end
endmodule

```

```

// data memory - handles load/store operations

module dmem (
    input logic clk,
    input logic reset,
    input logic [31:0] addr,
    input logic [31:0] write_data,
    input logic mem_read,
    input logic mem_write,
    output logic [31:0] read_data
);

    logic [31:0] memory [0:1023];

    // init memory for testing
    initial begin
        for (int i = 0; i < 1024; i++) begin
            memory[i] = 32'(i + 1);
        end
    end

    always_ff @(posedge clk) begin
        if (reset) begin
            for (int i = 0; i < 1024; i++) begin
                memory[i] <= 32'd0;
            end
        end else begin
            if (mem_write) begin
                memory[addr[11:2]] <= write_data;
            end
        end
    end

    always_ff @(posedge clk) begin
        if (mem_read) begin
            read_data <= memory[addr[11:2]];
        end else begin
            read_data <= 32'd0;
        end
    end
endmodule

```

5.9 Forwarding Unit

// forwarding unit - detects data hazards and generates forwarding control signals

```
module forwarding_unit (
    input logic [4:0] rs1_addr_ID_ExMem,
    input logic [4:0] rs2_addr_ID_ExMem,
    input logic [4:0] rs1_addr_IF_ID,
    input logic [4:0] rs2_addr_IF_ID,
    input logic [4:0] rd_WB,
    input logic      RegWr_WB,
    input logic      MemWr_ID_ExMem,
    input logic      MemWr_IF_ID,
    input logic      MemRd_ID_ExMem,
    input logic      MemRd_IF_ID,
    input logic      Beq_IF_ID,
    input logic      Bne_IF_ID,
    output logic ForwardA1,
    output logic ForwardB1,
    output logic ForwardA2,
    output logic ForwardB2,
    output logic ForwardC,
    output logic ForwardE
);

    always_comb begin
        if (RegWr_WB && (rd_WB != 5'b00000) && (rd_WB == rs1_addr_ID_ExMem)) begin
            ForwardA1 = 1'b1;
        end else begin
            ForwardA1 = 1'b0;
        end
    end

    always_comb begin
        if (RegWr_WB && (rd_WB != 5'b00000) && (rd_WB == rs2_addr_ID_ExMem)) begin
            ForwardB1 = 1'b1;
        end else begin
            ForwardB1 = 1'b0;
        end
    end

    always_comb begin
        if (RegWr_WB && (rd_WB != 5'b00000) && (rd_WB == rs1_addr_IF_ID) ||
            (RegWr_WB && MemWr_IF_ID && (rd_WB != 5'b00000) && (rd_WB == rs1_addr_IF_ID)) ||
            (RegWr_WB && (Beq_IF_ID || Bne_IF_ID) && (rd_WB != 5'b00000) && (rd_WB == rs1_addr_IF_ID))) begin
            ForwardA2 = 1'b1;
        end else begin
            ForwardA2 = 1'b0;
        end
    end

    always_comb begin
        if ((RegWr_WB && (rd_WB != 5'b00000) && (rd_WB == rs2_addr_IF_ID)) ||
            (RegWr_WB && MemWr_IF_ID && (rd_WB != 5'b00000) && (rd_WB == rs2_addr_IF_ID)) ||
            (RegWr_WB && (Beq_IF_ID || Bne_IF_ID) && (rd_WB != 5'b00000) && (rd_WB == rs2_addr_IF_ID))) begin
            ForwardB2 = 1'b1;
        end else begin
            ForwardB2 = 1'b0;
        end
    end

    always_comb begin
        if (RegWr_WB && MemWr_ID_ExMem && (rd_WB != 5'b00000) && (rd_WB == rs2_addr_ID_ExMem)) begin
            ForwardC = 1'b1;
        end else begin
            ForwardC = 1'b0;
        end
    end

    always_comb begin
        if ((RegWr_WB && MemRd_ID_ExMem && (rd_WB != 5'b00000) && (rd_WB == rs1_addr_ID_ExMem)) ||
            (RegWr_WB && MemWr_ID_ExMem && (rd_WB != 5'b00000) && (rd_WB == rs1_addr_ID_ExMem))) begin
            ForwardE = 1'b1;
        end else begin
            ForwardE = 1'b0;
        end
    end
endmodule
```

This forwarding unit detects data hazards in the pipeline and generates forwarding signals that supply the most recent values to dependent instructions without stalling the processor. The first two blocks, ForwardA1 and ForwardB1, handle EX-stage hazards by checking whether the instruction currently in the Write-Back (WB) stage writes a register that matches either rs1 or rs2 of the

instruction in the ID/EX stage. If so, the result from WB is forwarded directly to ALU input A or B. The next two blocks, ForwardA2 and ForwardB2, extend forwarding to the IF/ID stage and combine multiple cases including ALU operations, store instructions (sw), and branch instructions (beq/bne) by checking whether the WB destination register matches the source registers of the instruction in the IF/ID stage. This allows correct operand forwarding one cycle earlier, ensuring that branches and stores evaluate the right values. The ForwardC signal forwards the correct write-data for a store instruction when rs2 must be taken from a previous instruction still in WB. Finally, ForwardE forwards the base address register used by load and store instructions when the address dependency involves rs1. Collectively, these signals ensure that the datapath always uses the most recent values available in WB, eliminate unnecessary stalls, and support forwarding for ALU operations, memory accesses, and branch decisions.

5.10 Hazard Detection Unit

```
// hazard detection unit - detects load-use hazards and generates stall signals
module hazard_detection_unit (
    input logic [4:0] rd_ID_ExMem,
    input logic      RegWr_ID_ExMem,
    input logic      Beq_IF_ID,
    input logic      Bne_IF_ID,
    input logic [4:0] rs1_addr_IF_ID,
    input logic [4:0] rs2_addr_IF_ID,
    output logic PCenable,
    output logic IReenable,
    output logic nop
);

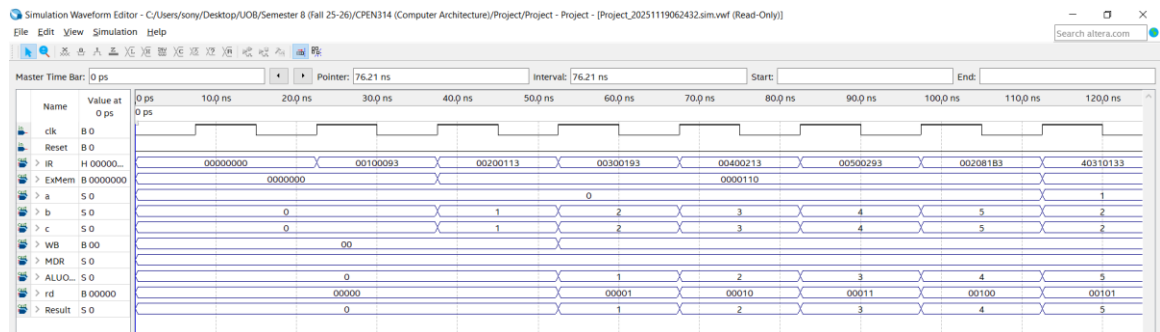
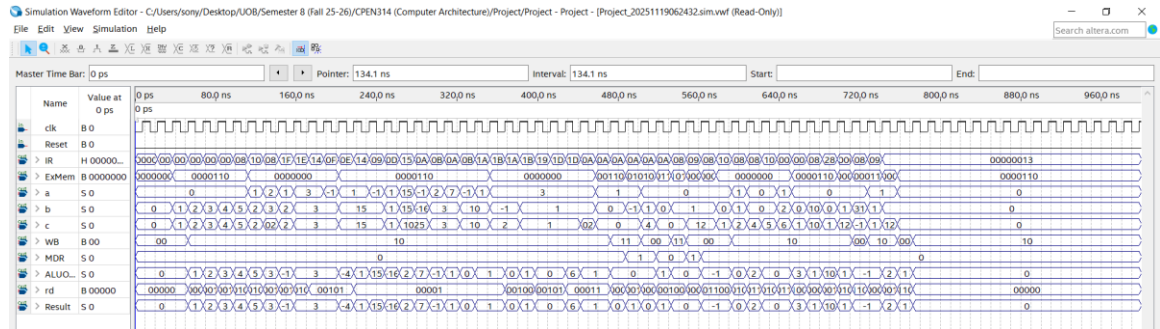
    always_comb begin
        // detect load-use hazard: LW in ExMem, result needed in IF/ID
        if (RegWr_ID_ExMem && (Beq_IF_ID || Bne_IF_ID) && (rd_ID_ExMem != 5'b00000) &&
            ((rd_ID_ExMem == rs1_addr_IF_ID) || (rd_ID_ExMem == rs2_addr_IF_ID))) begin
            PCenable = 1'b0;
            IReenable = 1'b0;
            nop = 1'b1;
        end else begin
            PCenable = 1'b1;
            IReenable = 1'b1;
            nop = 1'b0;
        end
    end
endmodule
```

This hazard detection unit identifies situations where a load-use or branch dependency requires stalling the pipeline to ensure correct execution. The logic checks whether the instruction in the EX/MEM stage is writing a register (RegWr_ID_ExMem) and whether the instruction currently in the IF/ID stage is a branch (beq or bne). If the EX/MEM instruction writes to a valid destination register (rd_ID_ExMem $\neq$ x0) and this destination matches either source register (rs1 or rs2) of the branch instruction in IF/ID, then the branch depends on a value that has not yet been written back. Because the branch comparator must evaluate its operands immediately in the ID stage, this creates a hazard. When such a dependency is detected, the control unit stalls the pipeline by disabling PCenable and IReenable (freezing both the PC and the IF/ID register) and asserting nop to insert a bubble in the next stage. If no hazard exists, all control signals remain enabled so the pipeline can proceed normally. This ensures that branch decisions are made using correct, fully updated

register values and prevents the processor from branching incorrectly due to stale operands.

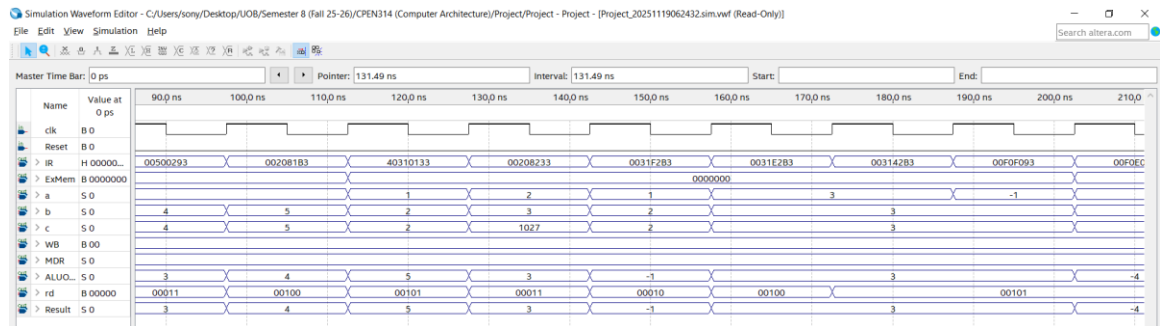
6 Simulation Results

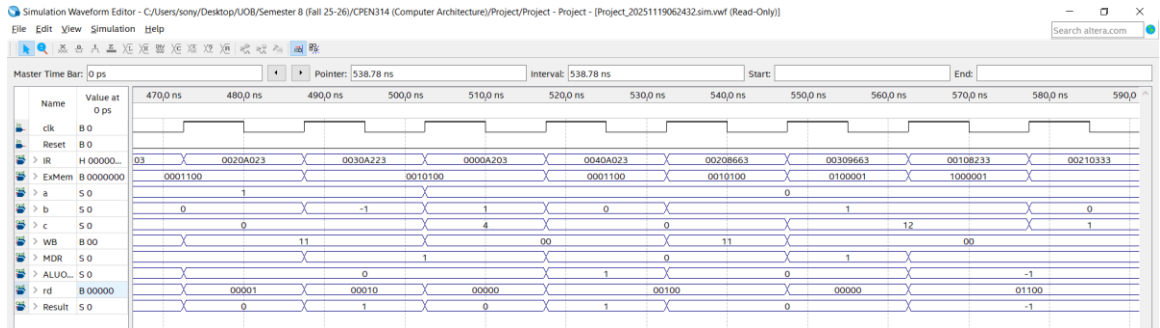
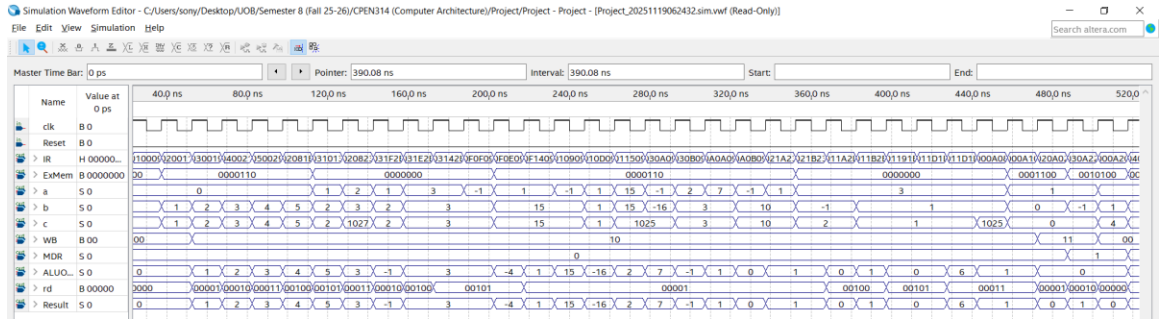
1. Without HDU and FU units:



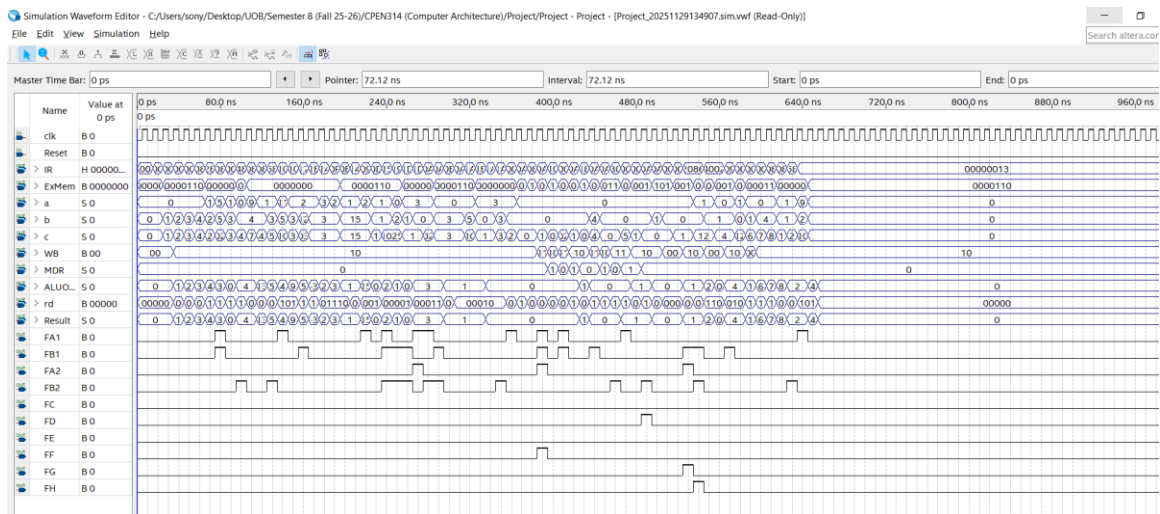
The first stage, Fetch, IR showed us 00100093, in the next stage we have A=0 (X0) and C=1 in the last line we see that result is equal to 1. We overwrite X1.

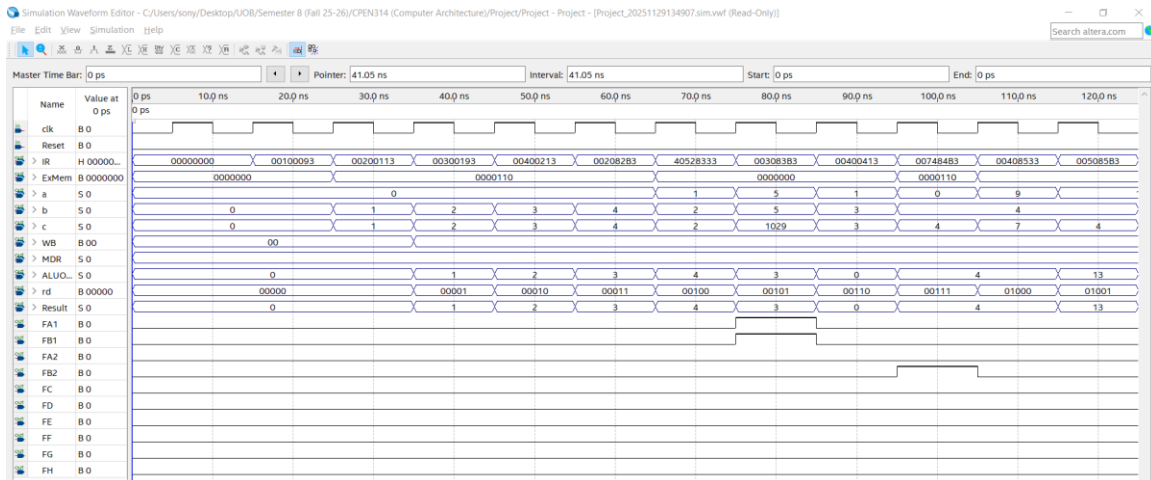
```
// basic ADDI tests
memory[0] = 32'h00100093; // ADDI x1, x0, 1 : x1 = 0 + 1 = 1 (overwrite initial x1=1)
memory[1] = 32'h00000013; // ADDI x3, x0, 3 : x3 = 0 + 3 = 3 (overwrite initial x3=3)
```





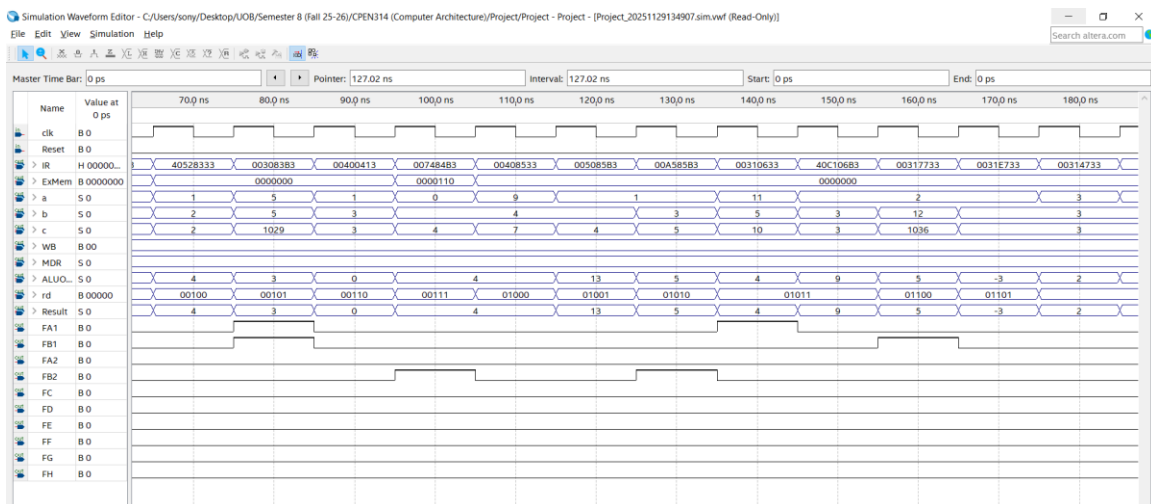
2. With HDU and FU units:





IR= 40528333 we have in the next stage A=5 (x5) and B=5 (x5) and C garbage because of the R-format. FA=1 and FB=1 (forwarding unit because there is dependency on rs1 and rs2). And the next stage is the WB stage which we wrote 0 on x5.

```
memory[5] = 32'h40528333; // SUB x6, x5, x5 : x6 = x5 - x5 = 0 (FORWARDING: x5 from MEM)
```



- Integrate a 7-segment or VGA output to display register or memory content during execution.
- Build a debugging unit that shows pipeline register content in real time.
- Synthesize the processor on an FPGA board for physical hardware validation.

8 Conclusion

In this project, we successfully designed and implemented a fully pipelined Mini-RISC-V processor capable of executing a selected subset of the RV32I instruction set. Through the integration of the five classical pipeline stages IF, ID, EX/ MEM, and WB, we demonstrated correct instruction flow and improved throughput compared to single-cycle architecture. Key architectural components, including the ALU, Control Unit, Immediate Generator, Register File, Data Memory, Forwarding Unit, and Hazard Detection Unit, were implemented and verified through waveform simulations. The processor was able to correctly handle arithmetic, logical, memory, and branch instructions while resolving hazards using a combination of forwarding, stalling, and flushing. Overall, this project provided hands-on experience with hardware design principles, pipeline behavior, and control logic, and it reinforced our understanding of how modern CPU architectures achieve efficiency and correctness through structured datapath and control designs.